# Programmed I/O

## 1 Basic Idea

**Definition**

**Programmed I/O** is an I/O method in which the CPU performs the transfer itself and repeatedly checks the device status until the device is ready.

- It is the simplest form of I/O.
- The CPU does all the active work.
- The OS sends one item of data to the device at a time.
- After each item, the CPU checks whether the device is ready for the next one.
- This repeated checking is called **polling** or **busy waiting**.

**Core Point**

In programmed I/O, the CPU is tied up until the I/O operation finishes.

## 2 Printing Example

**Scenario**

A user process wants to print the eight-character string `ABCDEFGH` on a printer through a serial interface.

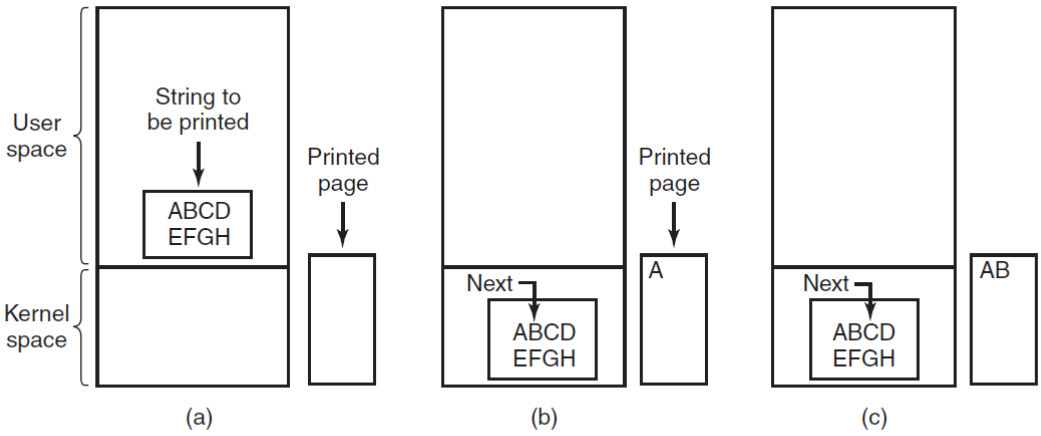


**Figure 5-7.** Steps in printing a string.

- The user process creates the string in a user-space buffer.

- The process makes a system call to open the printer for writing.
- If the printer is busy, the call may fail or block.
- The OS copies the string from the user buffer to a kernel buffer.
- The kernel buffer is safer for the OS to access while controlling the device.

## 3 Steps in Programmed I/O

1. Copy data from the user buffer into a kernel buffer.
2. Check whether the printer is available.
3. If the printer is unavailable, wait until it becomes available.
4. Copy the first character to the printer data register.
5. Check the printer status register.
6. If the printer is not ready, keep checking the status register.
7. When the printer becomes ready, copy the next character.
8. Repeat until all characters have been printed.
9. Return control to the user process.

**Status Register**

The printer status register tells the CPU whether the printer is ready to accept another character.

## 4 Pseudocode

**Writing a String Using Programmed I/O**

```
copy_from_user(buffer, p, count);     /* p is the kernel buffer */

for (i = 0; i < count; i++) {
    while (*printer_status_reg != READY)
        ;                             /* wait until printer is ready */

    *printer_data_register = p[i];    /* output one character */
}

return_to_user();
```

## 5 Polling and Busy Waiting

**Polling**

**Polling** means repeatedly reading a device status register to check whether the device is ready.

After sending one character, the CPU checks the printer status register. If the printer is not ready, the CPU checks again and continues this loop until the device becomes ready. During this waiting time, the CPU is not doing useful work for another process.

**Main Disadvantage**

Programmed I/O can waste CPU time because the CPU waits actively instead of sleeping or running another process.

## 6 When It Is Acceptable

| Case | Why programmed I/O may be acceptable |
|---|---|
| **Very fast device action** | If the device only copies data to an internal buffer, waiting time may be very short. |
| **Small transfers** | The overhead of interrupts or DMA may be larger than the transfer itself. |
| **Embedded systems** | If the CPU has nothing else to do, busy waiting may be simple and acceptable. |
| **Simple hardware** | Programmed I/O avoids extra interrupt or DMA complexity. |

## 7 When It Is Inefficient

- It is inefficient when the CPU has other useful work to run.
- It is inefficient for slow devices and long transfers.
- It does not overlap CPU computation with I/O activity.
- It can reduce overall system throughput in multiprogrammed systems.

**Better Alternatives**

For complex systems, interrupt-driven I/O or DMA is usually better because the CPU does not have to busy-wait for the whole transfer.

## 8 Quick Comparison

| Feature | Programmed I/O |
|---|---|
| **CPU role** | CPU directly controls the transfer. |
| **Waiting style** | CPU repeatedly polls the device. |
| **Hardware support** | Simple; no DMA controller is required. |
| **Main advantage** | Easy to understand and implement. |
| **Main disadvantage** | Wastes CPU time during busy waiting. |
| **Best use** | Small, fast, or embedded-system I/O. |